

13. MINIMAX ALGORITHM FOR GAMING

Aim:

To implement the Minimax Algorithm in Python for a two-player gaming environment and enable the computer to select the best possible move.

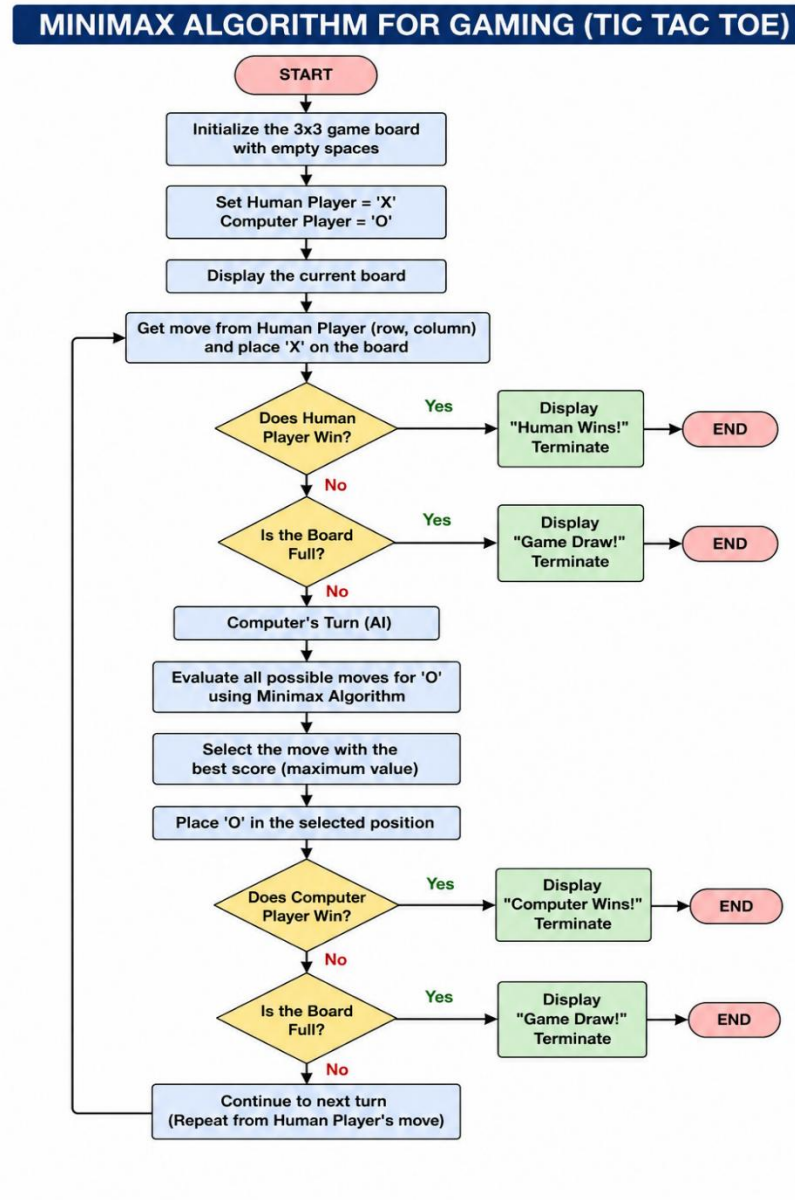
Objective :

- To understand the Minimax decision-making technique used in Artificial Intelligence.
- To implement optimal game-playing logic.
- To make the computer choose the best move by considering possible future game states.
- To demonstrate how AI can make decisions in a competitive game.

Algorithm:

1. Initialize the game board.
2. Define the winning conditions.
3. Check whether the current state is a terminal state.
4. If the AI wins, return a positive score.
5. If the human player wins, return a negative score.
6. If the game is a draw, return zero.
7. Generate all possible moves.
8. For the AI's turn, select the move with the maximum score.
9. For the human player's turn, select the move with the minimum score.
10. Continue recursively until a terminal state is reached.
11. Select the best move for the AI.
12. Display the final game result.

Flowchart :



Code:

```
def display_board(board):  
    print("\n")  
    for row in board:  
        print(" | ".join(row))  
        print("-" * 9)  
  
def check_winner(board, player):
```

```

for row in board:
    if all(cell == player for cell in row):
        return True
for col in range(3):
    if all(board[row][col] == player for row in range(3)):
        return True
if all(board[i][i] == player for i in range(3)):
    return True
if all(board[i][2 - i] == player for i in range(3)):
    return True
return False
def is_full(board):
    return all(cell != " " for row in board for cell in row)
def minimax(board, maximizing):
    if check_winner(board, "O"):
        return 1
    if check_winner(board, "X"):
        return -1
    if is_full(board):
        return 0
    if maximizing:
        best_score = -float("inf")
        for i in range(3):
            for j in range(3):
                if board[i][j] == " ":
                    board[i][j] = "O"

```

```

        score = minimax(board, False)

        board[i][j] = " "

        best_score = max(best_score, score)

    return best_score

else:

    best_score = float("inf")

    for i in range(3):

        for j in range(3):

            if board[i][j] == " ":

                board[i][j] = "X"

                score = minimax(board, True)

                board[i][j] = " "

                best_score = min(best_score, score)

    return best_score

def best_move(board):

    best_score = -float("inf")

    move = None

    for i in range(3):

        for j in range(3):

            if board[i][j] == " ":

                board[i][j] = "O"

                score = minimax(board, False)

                board[i][j] = " "

                if score > best_score:

                    best_score = score

                    move = (i, j)

```

```

    return move

def play_game():
    board = [[" " for _ in range(3)] for _ in range(3)]
    print("MINIMAX TIC TAC TOE")
    print("Human = X")
    print("Computer = O")
    while True:
        display_board(board)
        print("Your Turn")
        try:
            row = int(input("Enter row (0-2): "))
            col = int(input("Enter column (0-2): "))
        except ValueError:
            print("Enter numbers only.")
            continue
        if row not in range(3) or col not in range(3):
            print("Invalid position.")
            continue
        if board[row][col] != " ":
            print("Position already occupied.")
            continue
        board[row][col] = "X"
        if check_winner(board, "X"):
            display_board(board)
            print("You Win!")
            break

```

```
    if is_full(board):
        display_board(board)
        print("Game Draw!")
        break

    move = best_move(board)
    if move:
        board[move[0]][move[1]] = "O"
        print("\nComputer selected:", move)
    if check_winner(board, "O"):
        display_board(board)
        print("Computer Wins!")
        break
    if is_full(board):
        display_board(board)
        print("Game Draw!")
        break

play_game()
```

Output:

Output

MINIMAX TIC TAC TOE

Human = X

Computer = O

```
  |  |
-----
  |  |
-----
  |  |
-----
```

Your Turn

Enter row (0-2): 2

Enter column (0-2): 2

Output

```
X |  |
-----
  |  |
-----
  |  |
-----
```

Player O turn

Enter row (0-2): 1

Enter column (0-2): 0

```
X |  |
-----
O |  |
-----
  |  |
-----
```

Player X turn

```
Output

Enter row (0-2): 0
Enter column (0-2): 1

X | X |
-----
O |   |
-----
  |   |
-----
Player 0 turn
Enter row (0-2): 1
Enter column (0-2): 1
```

Result :

The Minimax Algorithm was successfully implemented in Python for a Tic Tac Toe gaming environment. The AI player evaluates possible future game states and selects the optimal move using maximization and minimization strategies. Thus, the program demonstrates intelligent decision-making in a competitive game.